Homework 1

Lucian Dorin Crainic - 1938430

Homework Description

In the final slide of the Block 1 introduction for the Advanced Software Engineering course, we are tasked with completing the implementation of a generic transaction execution system, initially demonstrated in Java. The goal is to model a transaction such as `transfer` using reusable components, including:

- A mapping between transaction names and the required steps (e.g., `transfer = ["deposit", "withdraw"]`)
- A second mapping describing how to extract arguments for each step from the full input argument list
- A method `execTrans` which interprets these mappings and executes each step in order, performing rollback if necessary

Python Implementation Strategy

The solution is implemented in Python using two core classes:

- `Account`: This class represents a bank account. Each account has a name and a balance, and supports depositing and withdrawing money. The operations are validated to avoid negative balances or negative transaction amounts.
- `Transaction`: This class is responsible for executing a high-level transaction based on a sequence of operation names and their argument mappings. It dynamically invokes operations using reflection (via `getattr`), and includes rollback logic to undo partial changes if a failure occurs during execution.

The transaction mappings are stored in-memory, and the rollback mechanism reverses any successful operation in the opposite order of their execution.

Account Class

Figure 1: Account Class

```python
class Account:
    def __init__(self, name: str, balance: float = 0.0):
        self.name = name
        self.balance = balance

    def deposit(self, amount: float) -> bool:
        if amount < 0:
            return False
        self.balance += amount
        return True

    def withdraw(self, amount: float) -> bool:
        if amount < 0 or self.balance < amount:
            return False
        self.balance -= amount
        return True

    def __str__(self):
        return f"{self.name}: {self.balance:.2f}"
```

Transaction Class

Figure 2: Transaction Class

```python
class Transaction:
    def __init__(self):
        self.descriptions: Dict[str, List[str]] = {}
        self.data: Dict[str, List[List[int]]] = {}

    def extract_args(self, args: List[Any], indices: List[int]) -> List[Any]:
        return [args[i] for i in indices]

    def prepare(self, operation_name: str, args: List[Any]) -> bool:
        method = getattr(args[0], operation_name, None)
        if method is None:
            return False
        return method(*args[1:])

    def rollback(self, done: List[tuple[str, List[Any]]]):
        print("ROLLBACK:", done)
        for op, args in reversed(done):
            if op == "deposit":
                args[0].withdraw(args[1])
            elif op == "withdraw":
                args[0].deposit(args[1])

    def commit(self, done: Dict[str, List[Any]]):
        print("COMMIT:", done)

    def exec_trans(self, trans_name: str, args: List[Any]) -> bool:
        done: List[tuple[str, List[Any]]] = []

        for op, index_set in zip(self.descriptions[trans_name], self.data[trans_name]):
            to_do_args = self.extract_args(args, index_set)
            success = self.prepare(op, to_do_args)
            if success:
                done.append((op, to_do_args))
            else:
                self.rollback(done)
                return False

        self.commit(dict(done))
        return True
```

Unit Tests

We wrote a comprehensive set of unit tests using Python's `unittest` module. Below are the major tests covered:

Test 1: Successful Transfer

Validates correct behavior when a customer with sufficient funds transfers money to another account.

Test 2: Insufficient Funds

Ensures the transaction is rejected if the source account lacks sufficient balance.

Test 3: Negative Deposit

Verifies the system prevents invalid operations like negative deposits.

Test 4: Atomic Transaction Partial Failure

Simulates a failure during a multi-step transaction and verifies that any successful operations prior to failure are rolled back.

Test 5: Atomic Transaction Complex Rollback

Tests a longer sequence of mixed operations and ensures the system performs a full rollback when the final step fails.

Test 6: All-or-Nothing Principle

Demonstrates that either all operations in a transaction succeed or none are applied. This test includes a successful chain followed by one that fails.

Test 7: Transaction Isolation Simulation

Emulates two transactions accessing the same shared account. Validates isolation and ensures that a second transaction does not succeed if the shared state is insufficient.

Test 8: Atomicity with Side Effects Verification

Creates a multi-actor transaction with expected rollback at the final stage. Confirms that no intermediate state affects any of the accounts involved.

Conclusion

This Python implementation of a model-based transaction system successfully mimics the transformation from use-case to execution, as envisioned in the original Java design. It:

- Executes steps based on descriptions and data mappings.
- Supports rollback to preserve consistency.
- Demonstrates correctness through 8 focused unit tests.

The resulting solution is modular, testable, and suitable for future extension.